LESSON 05 · MISSION GUIDE

The Scriptorium Scroll

Strings, files and CSVs: getting data in and out of Python without opening Excel.

From **Quartermistress Pell**
To **Jen, Keeper of the Guild Ledger**
By **Raven (a tired one)**

Keeper, it's the new moon, so the Scriptorium has sent The Scroll: their hand-copied version of the notice board. Forty quests. Quest numbers with and without the Q, regions in CAPITALS or cut short, danger as numbers, stars or "3/5", rewards with or without "cr", parties of "4" or "4 heroes". Every moon I fix it by hand in Excel, and every moon Excel turns the dates foreign.

Orrin says Python can read the scroll as it is, clean it, write a new one, and do exactly the same next moon. I'd like that before the Privy Council sits, because the Queen told them the Northern Wilds earn us the most and now somebody has to check. — P.

What you'll be able to do after today

- [ ] Take strings apart with `.split()` and put them back together with `.join()`
- [ ] Write text to a file and read it back, line by line or all at once
- [ ] Read a CSV with the `csv` module, as a list of dictionaries, one per row
- [ ] Clean a messy scroll with the string tools from lesson 01, in a loop, with functions
- [ ] Write a clean CSV that Excel, Captain Siv and next moon's script can all open
- [ ] Find and describe files with `pathlib`

Why this matters at work

- **Every analysis starts with a file.** Exports, extracts, downloads: they arrive as text, and somebody has to make sense of them. From today, that somebody is a script, and it never gets bored on row 37.
- **Excel changes your data when it opens it.** Leading zeros vanish, "01/02" becomes a date, long numbers become 1.2E+15. Python reads exactly what's there and nothing else.
- **The clean file is the product.** Once the scroll is clean, every later lesson (and the Charter Report) starts from it.

Four big ideas

1. `with open(...) as f`	Opens a file and closes it again automatically, even if something goes wrong in between. Always use it.
2. A file is just lines of text	Even a CSV. Each line ends in `"\n"`, and every value is text until you convert it.
3. The `csv` module does the fiddly bits	Commas inside quotes ("4,540 cr"), the header row, writing it all back out. Don't `.split(",")` a CSV yourself.
4. Paths are objects	`Path("data") / "quests.csv"` builds a path that works everywhere, and can list, read and check files.

Three things that bite on Windows

- Open CSVs with `newline=""`, or you get a blank line after every row you write.
- Add `encoding="utf-8"` whenever a file might contain a symbol or a name with an accent. Without it, Windows guesses, and guesses wrong.
- Paths print with backslashes (`my_work\report.txt`). Use forward slashes in your code; Python translates.

How today works

15 MIN
Lecture
Lars opens the scroll live and reads it three ways

60 MIN
Exercise sheet
Seven ravens, from strings to a finished clean file

15 MIN
Grimoire, page 5
Dalvric's moon tally. Three bugs. One typo

The ravens

#	FROM	RAVEN	YOU'LL USE
1	Orrin	Split and join	`.split() .join() set() sorted()`
2	The Queen	Write a file, read it back	`open("w") .write() .read() for line in f`
3	Pell	Open the scroll	`csv.DictReader fieldnames`
4	Pell	Clean it up	functions + `.strip() .upper() .replace()` + a lookup dict
5	Captain Siv	Write the clean file	`csv.DictWriter writeheader() writerows()`
6	The Queen	Crowns per region	dicts + `.get(key, 0)` + `max(key=)`
7 ⭐	Orrin	Files about files	`Path .glob() .read_text()`

Your three documents

GUIDE.pdf	This one.
EXERCISES.pdf	The exercise sheet, with the expected output to check against.
CHEATSHEET.pdf	Strings, files, csv and pathlib on one page. Keep it open.

You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if you're brave), each printing the expected output
- [ ] Dalvric's page prints the four expected lines, one commit per bug
- [ ] Everything is pushed to `lesson-05` and you've opened a pull request



Next lesson: The Full Moon. The Grimoire fails every full moon, and so will your script: the ledger has a quest whose reward says "n/a". You'll learn to read tracebacks properly, catch errors on purpose, and make scripts that survive real data.